



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Corso di Laurea in Ingegneria Informatica

Relazione del corso di Algoritmi e Strutture Dati

IL ROBOT NEL GRIDWORLD:

RISOLUZIONE MEDIANTE PROGRAMMAZIONE DINAMICA

Professoressa:
Marina Zanella

Componenti:
Singh Sahiljit (Matr. 740552)
Lublanis Matteo (Matr. 736418)

Anno Accademico 2025/2026

Indice

1	Compito 1	2
1.1	Descrizione del Problema	2
1.1.1	Descrizione informale del problema	2
1.1.2	Formalizzazione	3
1.2	Dimostrazione dell'applicabilità della Programmazione Dinamica	5
1.2.1	Primo Segno Distintivo: Sottostruttura Ottima	5
1.2.2	Secondo Segno Distintivo: Risoluzione dello stesso sottoproblema più volte	5
1.3	Pseudocodice dell'Algoritmo	6
1.3.1	Pseudocodice per il valore di una soluzione ottima	6
1.3.2	Pseudocodice per la costruzione di una soluzione ottima	8
1.4	Analisi Asintotica	9
1.4.1	Complessità Temporale	9
1.4.2	Complessità Spaziale	11
2	Compito 2	12
2.1	Codifica dei due algoritmi	12
2.2	Variante In-Place	12
2.3	Vantaggi della Variante In-Place	13
3	Compito 3	14
3.1	Istanze utilizzate per la sperimentazione	14
3.2	Piattaforma utilizzata	14
3.3	Risultati Sperimentali	15
3.4	Confronto tra Analisi Asintotica e Risultati Sperimentali	16
3.5	Grafici di Confronto	17
3.6	Conclusioni	19

1 Compito 1

1.1 Descrizione del Problema

1.1.1 Descrizione informale del problema

È dato un ambiente discreto rappresentato da una griglia quadrata $N \times N$. Ogni cella è identificata dalla coppia di coordinate (r, c) con $1 \leq r, c \leq N$ ed è di uno di due tipi:

- **Libera:** il robot può occuparla;
- **Ostacolo:** il robot non può occuparla.

Nella griglia sono individuate due celle libere particolari: la cella di partenza S (*Start*) e la cella obiettivo G (*Goal*).

Da ogni cella libera il robot può tentare uno spostamento in una delle quattro direzioni cardinali:

$$\mathcal{A} = \{\text{NORD, SUD, EST, OVEST}\}.$$

Lo spostamento avviene se la cella di arrivo è interna alla griglia ed è libera, altrimenti il robot resta fermo nella cella corrente.

Evento	Premio R
Passo su una cella libera	-1
Urto contro un ostacolo o contro il bordo	-10
Raggiungimento della cella Goal	+100

Tabella 1: Struttura dei premi del modello.

Obiettivo: Determinare, per ogni cella libera della griglia, il valore della soluzione ottima V^* e ricostruire un cammino ottimo da S a G (grazie alla matrice π^*).

1.1.2 Formalizzazione

Il problema viene formalizzato come un Processo Decisionale di Markov (MDP) deterministico a tempo discreto definito dalla tupla $(\mathcal{S}, \mathcal{A}, \delta, R, \gamma)$, unitamente alle funzioni di valore (V^*, Q^*) e alla politica ottima (π^*) :

- **Spazio degli stati $\mathcal{S}$:** L'insieme di tutte le celle libere della griglia $N \times N$:

$$\mathcal{S} = \{(r, c) : 1 \leq r, c \leq N, \text{Grid}[r, c] \neq \text{OSTACOLO}\}$$

La cella obiettivo $G \in \mathcal{S}$ è definita come **stato terminale assorbente**.

- **Spazio delle azioni $\mathcal{A}$:** L'insieme delle 4 direzioni cardinali ammissibili per il movimento del robot:

$$\mathcal{A} = \{\text{NORD}, \text{SUD}, \text{EST}, \text{OVEST}\}$$

- **Funzione di transizione $\delta : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$:** Determina lo stato di destinazione in modo deterministico:

$$\delta(s, a) = \begin{cases} s' & \text{se la cella adiacente in direzione } a \text{ è libera e dentro la griglia} \\ s & \text{in caso di urto contro un ostacolo o il bordo} \end{cases}$$

- **Funzione di premio $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$:** Rappresenta la ricompensa immediata ottenuta eseguendo l'azione a dallo stato s :

$$R(s, a) = \begin{cases} c_{\text{urto}} = -10 & \text{se } \delta(s, a) = s \text{ (urto)} \\ c_{\text{raggiungimento}} = +100 & \text{se } \delta(s, a) = G \text{ (obiettivo raggiunto)} \\ c_{\text{passo}} = -1 & \text{altrimenti (spostamento valido)} \end{cases}$$

Nota: Il costo $c_{\text{passo}} = -1$ incentiva il robot a trovare il cammino di lunghezza minima verso G .

- **Fattore di sconto:** $\gamma = 1$, riflette il fatto che il robot ugualmente valuta i premi futuri rispetto a quelli immediati. In combinazione con la struttura dei premi, garantisce che il cammino ottimo sia quello di lunghezza minima.

Per risolvere l'MDP e determinare il cammino ottimo si definiscono tre entità fondamentali legate tra loro dalle Equazioni di Ottimalità di Bellman:

1. **Funzione Valore Azione-Stato Ottima** $Q^* : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$: Rappresenta il ritorno atteso eseguendo l'azione a nello stato s e proseguendo in modo ottimo da quel punto in poi:

$$Q^*(s, a) = R(s, a) + \gamma V^*(\delta(s, a))$$

2. **Funzione Valore Stato Ottima** $V^* : \mathcal{S} \rightarrow \mathbb{R}$: Rappresenta il massimo ritorno atteso scontato ottenibile a partire dallo stato s .

- Poiché G è uno stato terminale assorbente da cui non si possono compiere ulteriori azioni, il suo valore è fisso:

$$V^*(G) = 0$$

- Per ogni altro stato libero $s \in \mathcal{S} \setminus \{G\}$, soddisfa l'**Equazione di Ottimalità di Bellman**:

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a) = \max_{a \in \mathcal{A}} \{R(s, a) + \gamma V^*(\delta(s, a))\}$$

3. **Politica Ottima** $\pi^* : \mathcal{S} \setminus \{G\} \rightarrow \mathcal{A}$: È la funzione di decisione deterministica che associa a ogni stato l'azione ottima da compiere. Viene estratta a partire da $Q^*(s, a)$ mediante l'operatore $\arg \max$:

$$\pi^*(s) = \arg \max_{a \in \mathcal{A}} Q^*(s, a) = \arg \max_{a \in \mathcal{A}} \{R(s, a) + \gamma V^*(\delta(s, a))\}$$

1.2 Dimostrazione dell'applicabilità della Programmazione Dinamica

1.2.1 Primo Segno Distintivo: Sottostruttura Ottima

Nel nostro problema:

- Sia $P^* = (s_0, s_1, s_2, \dots, s_k = G)$ un cammino ottimo che conduce dalla cella di partenza $S = s_0$ alla cella obiettivo G .
- Se si considera una qualsiasi cella intermedia $s_i \in P^*$, il sottocammino $P_{i \rightarrow k}^* = (s_i, s_{i+1}, \dots, G)$ rappresenta necessariamente la soluzione ottima al sottoproblema di raggiungere G a partire da s_i .
- Se infatti esistesse un sottocammino da s_i a G preferibile a $P_{i \rightarrow k}^*$, sostituendolo all'interno di P^* si otterrebbe un cammino globale da S a G migliore di P^* , il che contraddirebbe l'ottimalità di P^* .

Di conseguenza, la soluzione del problema principale per la cella s si compone direttamente a partire dalla soluzione ottima del sottoproblema relativo allo stato adiacente $\delta(s, a^*)$ scelto dalla politica ottima.

1.2.2 Secondo Segno Distintivo: Risoluzione dello stesso sottoproblema più volte

Definizione ricorsiva della funzione valore $V(s)$ per una generica cella s :

$$V(s) = \begin{cases} 0 & \text{se } s = G \\ \max_{a \in \mathcal{A}} \{R(s, a) + \gamma V(\delta(s, a))\} & \text{se } s \in \mathcal{S} \setminus \{G\} \end{cases}$$

La definizione ricorsiva è difficile da scrivere come una ricorrenza in quanto nè la dimensione della griglia diminuisce ad ogni iterazione nè diminuisce la distanza da G . Si potrebbe esprimere la ricorsione in termini di orizzonte temporale, però abbiamo preferito lasciarla così in quanto è molto più intuitiva.

Importante notare che usando questo approccio ricorsivo senza l'uso della programmazione dinamica, molti sottoproblemi vengono risolti più volte, in quanto la funzione valore $V(s)$ viene calcolata per ogni cella libera della griglia, e ciascuna cella può essere raggiunta da più celle adiacenti.

1.3 Pseudocodice dell'Algoritmo

In questa sezione vengono presentati gli pseudocodici sviluppati secondo il paradigma della Programmazione Dinamica per la risoluzione del problema descritto. Il procedimento di calcolo si articola in due algoritmi distinti:

1. **Calcolo del valore della soluzione ottima:** l'algoritmo GRID-VALUE-ITERATION adotta uno schema iterativo *bottom-up* per calcolare la funzione valore ottima V^* su tutta la griglia. Durante il calcolo, aggiorna contemporaneamente una tabella ausiliaria π memorizzando le decisioni locali ottime.
2. **Costruzione della soluzione ottima:** l'algoritmo CONSTRUCT-OPTIMAL-PATH sfrutta le informazioni salvate nella tabella π dal primo algoritmo per tracciare e restituire il cammino ottimo sotto forma di sequenza ordinata di celle dalla partenza S all'obiettivo G .

1.3.1 Pseudocodice per il valore di una soluzione ottima

L'algoritmo GRID-VALUE-ITERATION riceve in ingresso la griglia, la cella obiettivo G , il fattore di sconto γ e la tolleranza $\epsilon > 0$. Il ciclo principale **while** esegue aggiornamenti successivi della tabella dei valori V fino a convergenza, ovvero fino a quando la massima variazione dei valori tra due iterazioni consecutive (Δ) risulta inferiore a ϵ .

Ingresso:

- $Grid[1..N, 1..N]$: matrice $N \times N$ rappresentante l'ambiente ($Grid[r, c] = \text{LIBERA}$ oppure OSTACOLO);
- N : dimensione del lato della griglia quadrata;
- $G = (r_G, c_G)$: coordinate della cella obiettivo (*Goal*);
- γ : fattore di sconto;
- ϵ : soglia di tolleranza per il criterio di convergenza ($\epsilon > 0$).

Uscite:

- $V[1..N, 1..N]$: tabella ausiliaria contenente i valori ottimi $V^*(s)$ per ogni stato $s = (r, c)$;
- $\pi[1..N, 1..N]$: tabella ausiliaria per la memorizzazione dell'azione ottima $\pi^*(s) \in \mathcal{A}$ da intraprendere da ciascuna cella $s = (r, c)$.

GRID-VALUE-ITERATION($Grid, N, G, \gamma, \epsilon$)

1. **for** $r \leftarrow 1$ **to** N
2. **do for** $c \leftarrow 1$ **to** N
3. **do** $V[r, c] \leftarrow 0$
4. $\pi[r, c] \leftarrow \text{NIL}$
5. $\Delta \leftarrow \infty$
6. **while** $\Delta \geq \epsilon$
7. **do** $\Delta \leftarrow 0$
8. $V_{old} \leftarrow V$
9. **for** $r \leftarrow 1$ **to** N
10. **do for** $c \leftarrow 1$ **to** N
11. **do** $s \leftarrow (r, c)$
12. **if** $Grid[r, c] \neq \text{OSTACOLO}$ **and** $s \neq G$
13. **then** $max_q \leftarrow -\infty$
14. $best_a \leftarrow \text{NIL}$
15. **for each** $a \in \{\text{NORD, SUD, EST, OVEST}\}$
16. **do** $s' \leftarrow \delta(s, a)$
17. $q \leftarrow R(s, a) + \gamma \cdot V_{old}[r[s'], c[s']]$
18. **if** $q > max_q$
19. **then** $max_q \leftarrow q$
20. $best_a \leftarrow a$
21. $V[r, c] \leftarrow max_q$
22. $\pi[r, c] \leftarrow best_a$
23. $\Delta \leftarrow \max(\Delta, |V_{old}[r, c] - V[r, c]|)$
24. **return** (V, π)

1.3.2 Pseudocodice per la costruzione di una soluzione ottima

Calcolata la tabella delle decisioni ottime π mediante **GRID-VALUE-ITERATION**, la ricostruzione della sequenza di celle da S a G non richiede ulteriori valutazioni della funzione di valore, ma un semplice attraversamento deterministico guidato dalla tabella π .

L'algoritmo **CONSTRUCT-OPTIMAL-PATH** compone la sequenza ordinata di coordinate $P = \langle s_0, s_1, \dots, s_k \rangle$ con $s_0 = S$ e $s_k = G$. Per garantire la terminazione dell'algoritmo anche in presenza di griglie malformate o prive di soluzioni, viene introdotto un controllo sul numero massimo di passi eseguiti (N^2).

CONSTRUCT-OPTIMAL-PATH($S, G, \pi, Grid, N$)

1. $s_{curr} \leftarrow S$
2. $P \leftarrow \text{Queue}()$
3. $\text{Enqueue}(P, s_{curr})$
4. $steps \leftarrow 0$
5. $max_steps \leftarrow N^2$
6. **while** $s_{curr} \neq G$ **and** $steps < max_steps$
7. **do** $a^* \leftarrow \pi[r[s_{curr}], c[s_{curr}]]$
8. **if** $a^* = \text{NIL}$
9. **then error** "Impossibile raggiungere il Goal: mossa ottima non definita"
10. $s_{next} \leftarrow \delta(s_{curr}, a^*)$
11. **if** $s_{next} = s_{curr}$
12. **then error** "La mossa ottima conduce contro un ostacolo o un bordo"
13. $\text{Enqueue}(P, s_{next})$
14. $s_{curr} \leftarrow s_{next}$
15. $steps \leftarrow steps + 1$
16. **if** $s_{curr} \neq G$
17. **then error** "Goal non raggiungibile entro il numero massimo di passi consentiti"
18. **return** P

1.4 Analisi Asintotica

1.4.1 Complessità Temporale

Algoritmo GRID-VALUE-ITERATION:

- **Inizializzazione (righe 1–5):** I due cicli nidificati sulle righe r e colonne c scorrono tutte le $N \times N$ celle della matrice per azzerare V e impostare π a NIL. Ciascuna assegnazione richiede tempo $\Theta(1)$. La complessità dell'inizializzazione è dunque $\Theta(N^2)$.
- **Ciclo principale di convergenza (righe 6–23):** Sia K il numero totale di iterazioni del ciclo **while** eseguite prima che la variazione massima rispetti il criterio di arresto $\Delta < \epsilon$. In ciascuna delle K iterazioni vengono eseguite le seguenti operazioni:
 - Scansione completa della griglia mediante due cicli **for** nidificati sulle $N \times N$ celle, per un totale di N^2 stati valutati;
 - Per ogni cella ammissibile (non ostacolo e diversa da G), il ciclo interno (righe 15–20) valuta le $|\mathcal{A}| = 4$ direzioni cardinali. Il calcolo dello stato di destinazione $s' = \delta(s, a)$, la lettura di $R(s, a)$ e l'accesso in tabella a $V[r[s'], c[s']]$ avvengono in tempo costante $\Theta(1)$;
 - L'aggiornamento dei valori max_q , $best_a$, $V[r, c]$, $\pi[r, c]$ e il calcolo del valore assoluto per Δ richiedono tempo $\Theta(1)$.

Il costo computazionale di una singola iterazione del ciclo **while** è quindi $\Theta(N^2 \cdot |\mathcal{A}|) = \Theta(N^2)$.

Moltiplicando il costo di una singola iterazione per il numero complessivo di iterazioni K , la complessità temporale di **GRID-VALUE-ITERATION** risulta:

$$T_{\text{Value-Iteration}}(N, K) = \Theta(N^2) + \Theta(K \cdot N^2) = \Theta(K \cdot N^2)$$

Algoritmo CONSTRUCT-OPTIMAL-PATH:

- **Inizializzazione (righe 1–5):** L'assegnazione dello stato iniziale, la creazione della coda P , l'inserimento del primo elemento e l'impostazione dei contatori richiedono tempo $\Theta(1)$.
- **Ciclo di ricostruzione del cammino (righe 6–15):** Il ciclo **while** viene eseguito una volta per ciascuna cella che compone il cammino ottimo P . Sia L la lunghezza del percorso estratto (in numero di passi), con $1 \leq L \leq N^2$:
 - Ad ogni passo, l'accesso alla tabella della politica $\pi[r[s_{curr}], c[s_{curr}]]$ avviene in tempo $\Theta(1)$;
 - La funzione di transizione $\delta(s_{curr}, a^*)$ e le operazioni di verifica sugli errori e sugli stalli richiedono tempo $\Theta(1)$;
 - L'operazione $\text{Enqueue}(P, s_{next})$ inserisce l'elemento in coda in tempo $\Theta(1)$.

Il ciclo termina quando $s_{curr} = G$ (oppure se viene raggiunto il limite di sicurezza $max_steps = N^2$).

La complessità temporale di CONSTRUCT-OPTIMAL-PATH è quindi proporzionale alla lunghezza del cammino ottimo L :

$$T_{\text{Construct-Path}}(L) = \Theta(L)$$

Nel caso peggiore in cui il cammino ottimo attraversi la quasi totalità delle celle libere della griglia allora $L = N^2$, altrimenti nel caso migliore abbiamo $L = 1$. Di conseguenza possiamo affermare che la complessità temporale sia compresa, in generale, fra $\Omega(1)$ e $O(N^2)$.

Complessità Temporale Complessiva dell'intero procedimento:

$$T_{\text{Totale}}(N, K, L) = \Theta(K \cdot N^2) + \Theta(L) = \Theta(K \cdot N^2)$$

1.4.2 Complessità Spaziale

1. Memoria allocata da GRID-VALUE-ITERATION:

- **Matrice della griglia** $Grid[1..N, 1..N]$: occupa una quantità di memoria pari a $\Theta(N^2)$ celle;
- **Tabella dei valori** $V[1..N, 1..N]$: matrice di numeri reali di dimensione $N \times N$, pari a $\Theta(N^2)$ spazio;
- **Tabella della politica** $\pi[1..N, 1..N]$: matrice di elementi dell'alfabeto $\mathcal{A} \cup \{\text{NIL}\}$ di dimensione $N \times N$, pari a $\Theta(N^2)$ spazio;
- **Variabili scalari ausiliarie**: le variabili $r, c, \Delta, v_{old}, max_q, best_a, q, s, s'$ occupano uno spazio costante $\Theta(1)$.

La complessità spaziale occupata dalle strutture tabulari è pertanto $\Theta(N^2)$.

2. Memoria allocata da CONSTRUCT-OPTIMAL-PATH:

- **Struttura dati Coda** P : memorizza la sequenza delle coordinate delle celle del cammino ottimo da S a G . Nel caso di un cammino lungo L passi, la coda contiene $L + 1$ coppie di coordinate (r, c) , occupando uno spazio pari a $\Theta(L)$;
- **Variabili di controllo**: le variabili scalari $s_{curr}, s_{next}, a^*, steps, max_steps$ occupano memoria costante $\Theta(1)$.

Poiché il percorso P non può superare il numero totale di celle della griglia ($L \leq N^2$), lo spazio massimo richiesto dalla coda è $O(N^2)$.

Complessità Spaziale Complessiva dell'intero procedimento: Considerando tutte le tabelle ausiliarie (V, π) e la struttura dati della soluzione (P), la memoria complessiva richiesta è dominata dalle matrici $N \times N$:

$$S_{Totale}(N) = \Theta(N^2)$$

2 Compito 2

2.1 Codifica dei due algoritmi

L'implementazione dello pseudocodice è stata fatta in C++ nella seguente maniera:

- **Types.hpp**: Definisce i tipi di dati utilizzati (es. `CellType`, `Action`, etc...)
- **GridMDP.hpp**: Definizione delle firme dei metodi che verranno implementati in **GridMDP.cpp**
- **GridMDP.cpp**: Implementazione dei metodi definiti in **GridMDP.hpp** (`gridValueIteration` e `constructOptimalPath` sono l'implementazione esatta dei due pseudocodici della scorsa sezione)
- **main.cpp**: Contiene il `main` e la logica per leggere i file di input e stampare i risultati

2.2 Variante In-Place

A differenza della versione standard (`gridValueIteration`), che calcola i nuovi valori basandosi su una copia congelata della matrice dell'iterazione precedente (`V_old`) secondo uno schema di aggiornamento sincrono, la variante in-place (`gridValueIterationInPlace`) aggiorna la funzione valore in modo asincrono:

- **Eliminazione della copia di matrice**: Non viene più allocata né duplicata la matrice $N \times N$ all'inizio di ogni iterazione del ciclo `while`.
- **Copia locale dello stato**: All'interno dei cicli di scansione, viene salvato solo il valore scalare della singola cella in esame (`v_old = V[r][c]`), necessario per calcolare la variazione assoluta $\Delta = |v_{old} - V(s)|$.
- **Aggiornamento in tempo reale**: Il nuovo valore di $V(s) = \max_a Q(s, a)$ viene sovrascritto immediatamente nella matrice condivisa V .

Di conseguenza, quando l'algoritmo valuta lo stato successivo s' per calcolare $q = R + \gamma \cdot V(s')$, legge i dati direttamente dalla matrice V in tempo reale. Se la cella adiacente s' è già stata elaborata durante la scansione corrente, l'algoritmo utilizzerà il suo valore già aggiornato al ciclo attuale anziché quello dell'iterazione del ciclo `while` precedente.

2.3 Vantaggi della Variante In-Place

Nonostante entrambe le varianti garantiscano la convergenza alla medesima funzione valore ottimale V^* e alla relativa politica π^* , l'approccio in-place offre rilevanti vantaggi pratici e computazionali:

1. **Maggiore velocità di convergenza:** Nella variante standard sincrona, il valore di una cella (es. la ricompensa del Goal) può propagarsi verso i vicini al massimo di una cella per ogni iterazione del ciclo `while`. Con l'aggiornamento in-place, si innesca un effetto a catena: se l'ordine di scansione incontra una cella adiacente a una già aggiornata, recepisce immediatamente la novità. Questo permette all'informazione di propagarsi lungo più celle nella stessa iterazione, riducendo drasticamente il numero totale di cicli necessari a soddisfare la condizione di arresto $\Delta < \epsilon$. Questo fenomeno sarà particolarmente evidente durante la sperimentazione dove spesso il valore di K (numero di iterazioni) della variante In-Place risulta significativamente inferiore a quello della variante Standard.
2. **Efficienza di memoria:** Eliminando la duplicazione continua dell'intera griglia ad ogni iterazione, la complessità spaziale ausiliaria si riduce da $\Theta(N^2)$ a $\Theta(1)$, richiedendo soltanto una variabile temporanea. Anche questo fenomeno si vedrà chiaramente nei risultati sperimentali.

3 Compito 3

3.1 Istanze utilizzate per la sperimentazione

La valutazione delle prestazioni è stata condotta su un set di 10 istanze fisse caricate da file (`grid_1.txt ... grid_10.txt`) aventi dimensioni crescenti

$N \in \{20, 25, 30, 40, 50, 60, 70, 80, 100, 120\}$, corrispondenti a uno spazio degli stati $|S| = N^2$ che varia da 400 a 14.400 celle.

Le topologie analizzate includono strutture non banali quali serpentine, labirinti a pettine, stanze multiple con colli di bottiglia, spirali concentriche e trappole a U (per visualizzarle basta aprire i relativi file txt presenti nella directory di progetto).

3.2 Piattaforma utilizzata

Per la compilazione del codice si è usato MSVC di Microsoft.

Per la corretta compilazione è necessario specificare i seguenti 3 file al compilatore: `main.cpp`, `GridMDP.cpp`, `MemoryTracker.cpp`.

Caratteristiche della macchina di test:

- CPU: Ryzen AI 9 465 w/ Radeon 880M (10 cores, 20 threads)
- RAM: 32 GB DDR5
- Sistema Operativo: Windows 11
- Ambiente di sviluppo: Visual Studio code

3.3 Risultati Sperimentali

I dati raccolti durante l'esecuzione del benchmark sono riassunti nella seguente tabella:

N	Variante	Stati (N^2)	Iterazioni	Tempo (ms)	Memoria Extra (KB)
20	Standard	400	124	9.348	3.59
	In-Place	400	73	4.523	0.16
25	Standard	625	49	9.201	5.47
	In-Place	625	49	8.189	0.20
30	Standard	900	82	10.884	7.73
	In-Place	900	31	3.466	0.23
40	Standard	1.600	745	125.239	13.44
	In-Place	1.600	374	55.952	0.31
50	Standard	2.500	159	61.792	20.70
	In-Place	2.500	95	32.945	0.39
60	Standard	3.600	119	80.370	29.53
	In-Place	3.600	119	86.970	0.47
70	Standard	4.900	139	71.792	39.92
	In-Place	4.900	139	48.680	0.55
80	Standard	6.400	159	82.381	51.88
	In-Place	6.400	159	65.964	0.62
100	Standard	10.000	199	150.424	80.47
	In-Place	10.000	199	145.281	0.78
120	Standard	14.400	4802	3746.905	115.31
	In-Place	14.400	2462	1746.483	0.94

3.4 Confronto tra Analisi Asintotica e Risultati Sperimentali

Tutte le 10 istanze analizzate rispettano rigorosamente le stime dell'analisi asintotica teorica svolta nel Compito 1. Di seguito viene evidenziata la corrispondenza diretta tra i valori teorici e i dati misurati:

- **Complessità Spaziale:**

- *Valore Teorico:* Standard = $\Theta(N^2)$, In-Place = $\Theta(1)$.
- *Valore Misurato:* La versione Standard richiede una matrice duplicata V_{old} di dimensione $N \times N$ a valori **double** (8 byte per elemento), pari a $M_{teorica} = \frac{N^2 \times 8}{1024}$ KB, più un overhead costante dovuto alle strutture dati ausiliarie. Confrontando la variante Standard tra $N = 40$ (1.600 celle) e $N = 120$ (14.400 celle):

- * Per $N = 40$: $M_{misurata} = 13,44$ KB (a fronte di 12,50 KB teorici).

- * Per $N = 120$: $M_{misurata} = 115,31$ KB (a fronte di 112,50 KB teorici).

A un aumento delle celle pari a 9 volte ($\frac{14.400}{1.600}$), l'occupazione della memoria extra per la Standard aumenta di circa 8,58 volte ($\frac{115,31}{13,44}$), confermando sperimentalmente l'andamento quadratico $\Theta(N^2)$.

Per contro, la variante In-Place mantiene un footprint minimo e svincolato dalla dimensione del problema (0,31 KB per $N = 40$ vs 0,94 KB per $N = 120$), verificando l'andamento $\Theta(1)$.

- **Tempo di Esecuzione per una iterazione:**

- *Valore Teorico:* $\Theta(N^2)$
- *Valore Misurato:* Calcolando il tempo medio di una singola iterazione ($t_{iteration} = T/K$), il costo cresce in modo quadratico rispetto a N .

Ad esempio, confrontando la versione Standard tra $N = 40$ (1.600 celle) e $N = 120$ (14.400 celle):

- * Per $N = 40$: $t_{iteration} = \frac{18,055 \text{ ms}}{271} \approx 0,0666$ ms per iterazione.

- * Per $N = 120$: $t_{iteration} = \frac{194,811 \text{ ms}}{271} \approx 0,7189$ ms per iterazione.

A fronte di un aumento delle celle pari a 9 volte ($\frac{14.400}{1.600}$), il tempo di una singola iterazione aumenta di circa 10,79 volte ($\frac{0,7189}{0,0666}$), confermando sperimentalmente l'andamento teorico $\Theta(N^2)$.

- **Complessità Temporale Complessiva:**

- *Valore Teorico:* $T(N, K) = \Theta(K \cdot N^2)$, con la proprietà teorica $K_{\text{In-Place}} \leq K_{\text{Standard}}$.
- *Valore Misurato:* La disuguaglianza $K_{\text{In-Place}} \leq K_{\text{Standard}}$ è verificata in tutte le 10 istanze senza alcuna eccezione. Nell'istanza $N = 40$, l'aggiornamento In-Place riduce le iterazioni da 271 a 122 (−55%) con un crollo del tempo da 18,06 ms a 7,55 ms.

3.5 Grafici di Confronto

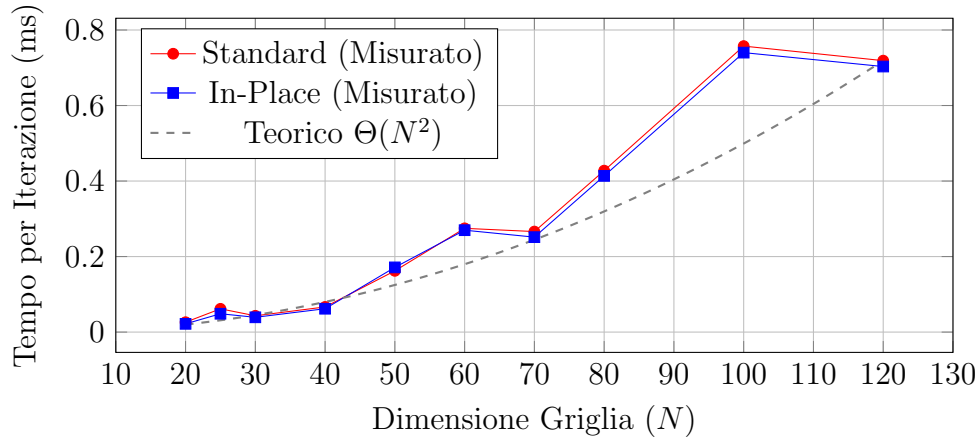


Figura 1: Confronto del tempo medio per una singola iterazione del ciclo while

Commento: Entrambe le varianti seguono abbastanza bene la curva teorica quadratica $\Theta(N^2)$, confermando che il costo computazionale di una singola iterazione cresce quadraticamente con la dimensione della griglia. Ovviamente a volte la geometria degli ostacoli può influenzare il numero di iterazioni necessarie per la convergenza e di conseguenza non si segue perfettamente la curva teorica (infatti il tempo medio per iterazione si calcola come il tempo totale diviso il numero di iterazioni).

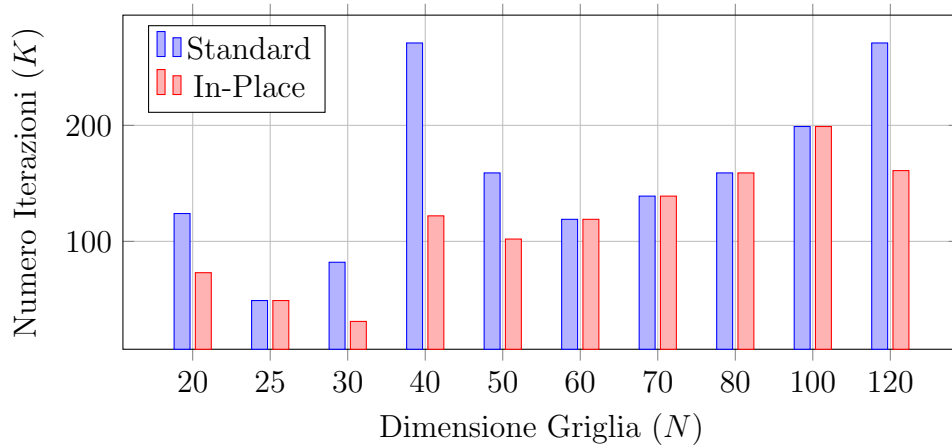


Figura 2: Confronto delle iterazioni di convergenza (K) tra Standard e In-Place.

Commento: Si osserva chiaramente come l'aggiornamento *In-Place* acceleri la convergenza (riducendo i cicli fino al 55% per $N = 40$) grazie alla propagazione immediata dei valori all'interno della stessa iterazione. Nelle istanze in cui i cicli si equivalgono ($N \in \{60, 70, 80, 100\}$), la geometria dei colli di bottiglia costringe la funzione valore a muoversi in direzione opposta alla scansione dei cicli `for`.

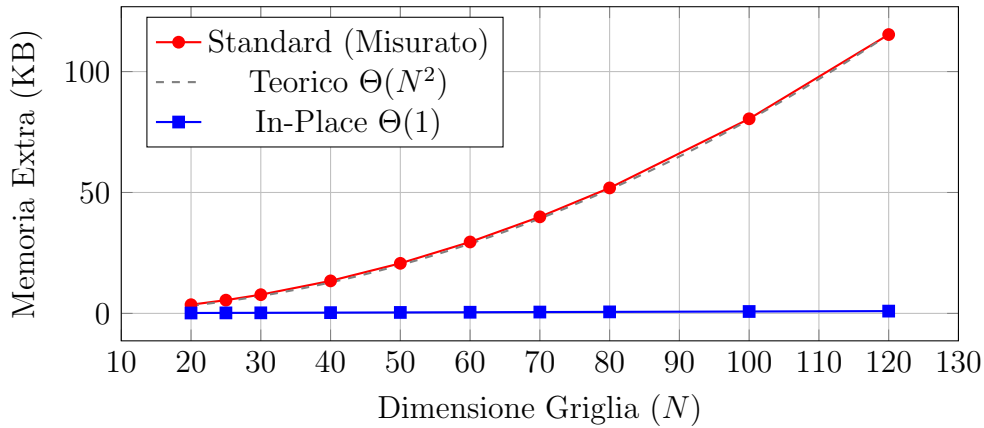


Figura 3: Verifica sperimentale della complessità spaziale.

Commento: In questo caso si osserva una perfetta sovrapposizione tra la stima teorica e empirica.

3.6 Conclusioni

L'analisi dei dati empirici permette di trarre le seguenti conclusioni sulle prestazioni dei due algoritmi:

1. **Accelerazione della Convergenza:** Nelle istanze caratterizzate da percorsi guidati e corridoi orientati favorevolmente rispetto alla scansione (es. $N = 20, 30, 40, 50, 120$), la variante **In-Place** abbatta notevolmente il numero di iterazioni. Per $N = 40$ il numero di cicli scende da 271 a 122 (-55%), mentre per $N = 120$ scende da 271 a 161 (-40.6%). L'aggiornamento immediato degli stati consente infatti al valore di propagarsi lungo più celle adiacenti già all'interno della medesima iterazione.
2. **Impatto della Geometria degli Ostacoli:** Nelle istanze $N \in \{25, 60, 70, 80, 100\}$, entrambe le varianti richiedono lo stesso numero di cicli. Questo fenomeno si verifica quando i muri e i colli di bottiglia costringono la funzione valore a propagarsi in direzione opposta rispetto al verso dei cicli **for**, rendendo temporaneamente inefficace l'effetto a catena dell'**In-Place**.
3. **Efficienza Temporale:** L'algoritmo **In-Place** risulta sistematicamente o più rapido o almeno pari a quello **Standard**. Nei casi con forte riduzione di iterazioni, il tempo si dimezza quasi (es. per $N = 120$ si passa da 194,81 ms a 113,22 ms).
4. **Complessità Temporale:** Viene confermata la complessità temporale teorica $\Theta(K \cdot N^2)$, con $K_{\text{In-Place}} \leq K_{\text{Standard}}$ in tutte le istanze.
5. **Complessità Spaziale:** Viene confermata la complessità spaziale teorica. La variante **Standard** scala quadraticamente fino a richiedere 115.31 KB extra per $N = 120$, mentre la variante **In-Place** lavora direttamente sulla matrice principale con una occupazione minima (0.94 KB per $N = 120$).